

Validation results

Tags

System prompt:

You are a helpful assistant that can assist with creating behavior trees.

Your task:

- Convert the provided summary of a behavior into an XML-formatted behavior tree.
- Ensure the behavior tree matches the description in the summary.
- The behavior tree must be compatible with the BehaviorTree.CPP library.
- Only use the actions and parameters provided in the action list below the summary.

Output Requirements:

- Output only the XML representation of the behavior tree. Do not include explanations, comments, or any additional text.
- Ensure all actions and parameters strictly match the provided list.
- If possible limit the use of SubTrees.

Please generate the behavior tree based on the summary and action list provided.

Task 1: Warehouse Management

This behavior tree manages a mobile robot in a warehouse. It must identify and scan inventory using a barcode reader. If the barcode is valid, it updates the inventory database in real time, if it's invalid, it picks the object and brings it to the security department for further investigation. Then, it checks the battery status, if it's below 10% it goes back to the charging station.

Actions and Conditions: [FindObject (parameters: object), Scan (parameters: barcode), isBarcodeValid, UpdateInventory (parameters: object, barcode), Pick (parameters: object), MoveTo (parameters: location), isBatteryCharged, Charge]

Reference

```
<root BTCPP_format="4" main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Fallback>
        <Sequence>
          <FindObject object="{object}" />
          <Scan barcode="{barcode}" />
          <isBarcodeValid />
          <UpdateInventory object="{object}" barcode="{barcode}" />
        </Sequence>
        <Sequence>
          <Pick object="{object}" />
          <MoveTo location="Security Department"/>
        </Sequence>
      </Fallback>
    </Sequence>
    <Sequence>
      <isBatteryCharged />
      <MoveTo location="Charging Station" />
    </Sequence>
  </BehaviorTree>
</root>
```

Ours (4.0s)

```
<root BTCPP_format="4" main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Fallback>
        <Sequence>
          <FindObject object="{object}" />
          <Scan barcode="{barcode}" />
          <isBarcodeValid />
          <UpdateInventory object="{object}" barcode="{barcode}" />
        </Sequence>
        <Sequence>
          <FindObject object="{object}" />
          <Pick object="{object}" />
          <MoveTo location="Security Department" charge="true" />
        </Sequence>
      </Fallback>
    </Sequence>
    <Sequence>
      <isBatteryLow threshold="10" />
      <MoveTo location="Charging Station" />
    </Sequence>
  </BehaviorTree>
</root>
```

```
</BehaviorTree>
</root>
```

CFR: 2/1=4

BTGenBot (1m 11s)

```
<root main_tree_to_execute = "MainTree" >

  <BehaviorTree ID="MainTree">
    <Sequence>
      <Fallback name="root_Fallback">
        <Sequence name="scan_inventaire">
          <Action ID="ScanBarcode" goal="123"/>
          <Action ID="UpdateInventaire" produits="{inventaire}" entrees="{inventaire}"/>
        </Sequence>
        <Action ID="PickObject" nom_objet="{object_name}"/>
        <Action ID="BringToSecurity" objet="{object_name}"/>
        <Action ID="Charging" percent_batterie="10" etat_batterie="déchargé"/>
      </Fallback>
    </Sequence>
  </BehaviorTree>

  <TreeNodesModel>
    <Action ID="Charge">
      <input_port default="10" name="percent_batterie">Percentage of battery to charge</input_port>
      <input_port default="déchargé" name="etat_batterie">Battery state</input_port>
    </Action>
    <Action ID="Charging">
      <input_port default="10" name="percent_batterie">Percentage of battery to charge</input_port>
      <input_port default="déchargé" name="etat_batterie">Battery state</input_port>
    </Action>
    <Action ID="UpdateInventaire">
      <input_port default="inventaire" name="produits">Produits to update</input_port>
      <input_port default="inventaire" name="entrees">Entries to update</input_port>
    </Action>
    <Action ID="ScanBarcode">
      <input_port default="123" name="goal">Target object</input_port>
    </Action>
    <Action ID="PickObject">
      <input_port default="robot" name="nom_objet">Object name</input_port>
    </Action>
  </TreeNodesModel>

</root>
```

CFR: 0/1= 0

o3 (15s)

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="main_sequence">
      <FindObject object="{object}" />
      <Scan barcode="{barcode}" />

      <Fallback name="barcode_handling">
        <Sequence name="valid_barcode">
          <isBarcodeValid />
          <UpdateInventory object="{object}" barcode="{barcode}" />
        </Sequence>
        <Sequence name="invalid_barcode">
          <Pick object="{object}" />
          <MoveTo location="security_department" />
        </Sequence>
      </Fallback>

      <Fallback name="battery_check">
        <isBatteryCharged />
        <Sequence name="charge_sequence">
          <MoveTo location="charging_station" />
          <Charge />
        </Sequence>
      </Fallback>
    </Sequence>
  </BehaviorTree>
</root>
```

CFR: 1/2 = 0.5

Task 2: Hospital Service and Emergency Response

This behavior tree manages a service robot operating in a hospital environment, it performs two tasks in parallel: organizing storage and monitoring patient vital signs. In storage, it identifies misplaced objects, picks them up, and places them in their correct locations. Simultaneously, it checks a patient's vital signs. If a critical condition is detected, it sends an emergency alert to the staff and waits for their arrival. Once the staff arrives, it moves to storage, picks the requested medicine, and continues operations.

Actions: [isMisplaced, Pick (parameters: object), Place(parameters: object, location), checkVitalSigns(parameters: patient_id, room_id), isCritical, SendAlert (parameters: alert_type), MoveTo (parameters: location)]

Reference

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Parallel success_count="2" failure_count="1">
      <Sequence>
        <isMisplaced/>
        <Pick object="{object}" />
        <Place object="{object}" location="storage"/>
      </Sequence>
      <Sequence>
        <checkVitalSigns patient_id="patient_id" room_id="room_id"/>
        <Fallback>
          <Sequence>
            <isCritical/>
            <SendAlert alert_type="critical"/>
            <WaitForStaff/>
          </Sequence>
          </Fallback>
          <MoveTo location="storage"/>
          <Pick object="medicine"/>
          <Place object="medicine" location="room_id"/>
        </Sequence>
      </Parallel>
    </BehaviorTree>
  </root>
```

Ours (5.5s)

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Parallel success_count="2" failure_count="1">
      <Sequence>
        <isMisplaced/>
        <Pick object="{object}" />
        <Place object="{object}" location="storage"/>
      </Sequence>
      <Sequence>
        <checkVitalSigns patient_id="Rohn" room_id="RoomA"/>
        <Fallback>
          <Sequence>
            <isCritical/>
            <SendAlert alert_type="critical_type"/>
            <WaitForStaff/>
          </Sequence>
          </Fallback>
          <MoveTo location="storage"/>
          <Pick object="med"/>
        </Sequence>
      </Parallel>
    </BehaviorTree>
  </root>
```

CFR: 3/1 = 3

BTGenBot (1m 16s)

```
<root main_tree_to_execute = "MainTree" >
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Action ID="MoveRobot" goal="{object}" />
      <Action ID="UseBarcodeReader" barcode="{barcode}" />
      <Action ID="GetResult" result="{result}" />
      <Fallback>
        <Condition ID="isBarcodeValid" barcode="{barcode}" />
        <Sequence>
          <Action ID="Pick" object="{object}" />
```

```

        <Action ID="MoveTo" location="{destination}" isBatteryCharged="{isBatteryCharged}" Charge="{charge}"/>
    </Sequence>
</Fallback>
<Action ID="WriteToCSV" file="inventory.csv" action="{action}" object="{object}" barcode="{barcode}"/>
</Sequence>
</BehaviorTree>
</root>

```

CFR: 1/1=1

o3 (25s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Parallel success_threshold="2" failure_threshold="1">
      <Repeat num_cycles="-1">
        <Sequence>
          <isMisplaced/>
          <Pick object="{object}"/>
          <Place object="{object}" location="{correct_location}"/>
        </Sequence>
      </Repeat>

      <Repeat num_cycles="-1">
        <Sequence>
          <checkVitalSigns patient_id="{patient_id}" room_id="{room_id}"/>
          <Fallback>
            <Sequence>
              <isCritical/>
              <SendAlert alert_type="emergency"/>
              <MoveTo location="storage"/>
              <Pick object="requested_medicine"/>
            </Sequence>
            <AlwaysSuccess/>
          </Fallback>
        </Sequence>
      </Repeat>
    </Parallel>
  </BehaviorTree>
</root>

```

CFR: 2/1 = 2

Task 3: Restaurant Order Management

This behavior tree manages an autonomous robot that handles orders in a restaurant. It continuously checks if one table is ready to order. If it's already serving a table it adds the order to the queue and manages it later. Otherwise, it moves directly to the table to execute the order. During an order, the food ordered is annotated until the clients are satisfied with their order. Then, if the queue is empty it goes back to the initial spot, otherwise, it goes to the next table in the queue.

Actions: [MoveTo (parameters: location), isReadyToOrder, TakeOrder(parameters: table, food, beverage), addToQueue (parameters: table), removeFromQueue (parameters: table), isOrderFinished, isQueueNotEmpty]

Reference

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="OrderTree">
    <Sequence>
      <RepeatUntilSuccess>
        <Fallback>
          <Sequence>
            <isReadyToOrder />
            <MoveTo location="{table}"/>
            <TakeOrder table="{table}" food="{food}" beverage="{beverage}"/>
            <isOrderFinished />
            <removeFromQueue table="{table}"/>
          </Sequence>
          <addToQueue table="{table}"/>
        </Fallback>
      </RepeatUntilSuccess>
      <Fallback>
        <Sequence>
          <isQueueNotEmpty />
          <MoveTo location="next_table"/>
        </Sequence>
        <MoveTo location="initial_spot"/>
      </Fallback>
    </Sequence>
  </BehaviorTree>
</root>

```

```

    </Sequence>
  </BehaviorTree>
</root>

```

Ours (4.3s)

```

<root main_tree_to_execute="OrderTree" BTCPP_format="4">
  <BehaviorTree ID="OrderTree">
    <Sequence>
      <RepeatUntilSuccess>
        <Fallback>
          <ForceSuccess>
            <Sequence>
              <isReadyToOrder name="isReadyToOrder"/>
              <MoveTo name="move_to_table" location="table"/>
              <AddToQueue name="add_to_queue" table="table" food="pizza" beverage="water"/>
            </Sequence>
          </ForceSuccess>
          <MoveTo name="move_to_table" location="table"/>
        </Fallback>
      </RepeatUntilSuccess>
      <Fallback>
        <MoveTo name="go_to_next_table" location="next_table"/>
        <MoveTo name="go_to_initial_spot" location="initial_spot"/>
      </Fallback>
    </Sequence>
  </BehaviorTree>
</root>

```

CFR: 1/2 = 0.5

BTGenBot (1m 26s)

```

<root main_tree_to_execute="BehaviorTree">

  <BehaviorTree ID="BehaviorTree">
    <Fallback>
      <Sequence>
        <Fallback>
          <Condition ID="IsTableReadyToOrder" />
          <Action ID="AddToQueue" queue_name="my_queue" order="{order}" />
          <Action ID="ManageOrder" queue_name="my_queue" order="{order}" />
        </Fallback>
        <Fallback>
          <Condition ID="IsTableReadyToEat" />
          <Action ID="ExecuteOrder" order="{order}" />
        </Fallback>
      </Sequence>
      <Action ID="GoToInitialLocation" />
      <Action ID="GoToNextTableInQueue" queue_name="my_queue" />
    </Fallback>
  </BehaviorTree>

  <TreeNodeModel>
    <Action ID="AddOrderToQueue">
      <input_port name="queue_name" type="std::string">Queue name</input_port>
      <input_port name="order" type="std::string">Order</input_port>
    </Action>
    <Action ID="ExecuteOrder">
      <input_port name="order" type="std::string">Order</input_port>
    </Action>
    <Action ID="GoToInitialLocation">
    </Action>
    <Action ID="GoToNextTableInQueue">
      <input_port name="queue_name" type="std::string">Queue name</input_port>
    </Action>
    <Condition ID="IsTableReadyToEat">
    </Condition>
    <Condition ID="IsTableReadyToOrder">
    </Condition>
  </TreeNodeModel>

</root>

```

CFR: 2/3= 0.66

o3 (40s)

```

<root main_tree_to_execute="RestaurantOrderManager">
  <BehaviorTree ID="RestaurantOrderManager">
    <Fallback name="RootSelector">
      <Sequence name="ContinueCurrentOrder">
        <Inverter>
          <Condition ID="isOrderFinished"/>
        </Inverter>
        <Fallback name="HandleCurrentOrAddQueue">
          <Sequence name="QueueNewTable">
            <Condition ID="isReadyToOrder"/>
            <Action ID="addToQueue" table="{ready_table}"/>
          </Sequence>
          <Action ID="TakeOrder" table="{current_table}" food="{food}" beverage="{beverage}"/>
        </Fallback>
      </Sequence>

      <Sequence name="ServeNextInQueue">
        <Condition ID="isQueueNotEmpty"/>
        <Action ID="removeFromQueue" table="{next_table}"/>
        <Action ID="MoveTo" location="{next_table}"/>
      </Sequence>

      <Sequence name="GotoReadyTable">
        <Condition ID="isReadyToOrder"/>
        <Action ID="MoveTo" location="{ready_table}"/>
      </Sequence>

      <Sequence name="ReturnToBase">
        <Condition ID="isOrderFinished"/>
        <Inverter>
          <Condition ID="isQueueNotEmpty"/>
        </Inverter>
        <Action ID="MoveTo" location="initial_spot"/>
      </Sequence>
    </Fallback>
  </BehaviorTree>
</root>

```

CFR: 6/2=3

Task 4: Search and Rescue Exploration

This behavior tree handles a robot that explores the environment to locate survivors. If it detects one, it halts exploration, sends a real-time status update to the command, and waits for the rescue team. Once the staff arrives, it resumes exploration. However, if an obstruction is encountered during exploration, the robot must attempt to navigate around it. If rerouting fails, it sends an alert and returns to a predefined safe location before attempting exploration again.

Actions: [MoveTo (parameters: location), foundSurvivor, Alert(parameters: alert_type), WaitStaff, hasStaffArrived, isObstructed]

Reference

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Fallback>
      <Sequence>
        <MoveTo location="area"/>
        <foundSurvivor />
        <Alert alert_type="survivor"/>
        <WaitStaff />
        <hasStaffArrived />
      </Sequence>
      <Sequence>
        <isObstructed />
        <Fallback>
          <MoveTo location="re-route"/>
          <Alert alert_type="routing" />
        </Fallback>
      </Sequence>
    </Fallback>
  </BehaviorTree>
</root>

```

Ours (3.3s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Fallback name="ExploreOrRescue">

```

```

    <Sequence>
      <MoveTo location="ExploreArea"/>
      <Action ID="foundSurvivor"/>
      <Action ID="Alert" alert_type="alert"/>
      <Action ID="WaitStaff"/>
    </Sequence>
    <Sequence>
      <Action ID="hasStaffArrived"/>
      <Action ID="isObstructed"/>
      <Fallback>
        <Action ID="MoveTo" location="Rerouting"/>
        <Action ID="Alert" alert_type="alert"/>
      </Fallback>
    </Sequence>
  </Fallback>
</BehaviorTree>
</root>

```

CFR: 2/2=1

BTGenBot (45s)

```

<root main_tree_to_execute = "MainTree">
  <BehaviorTree ID="MainTree">
    <Fallback>
      <Sequence>
        <Action ID="foundSurvivor"/>
        <Sequence>
          <Action ID="HaltExploration"/>
          <Action ID="SendRTStatusUpdate" command="{location}">
            <input_port default="location" name="command">
              </input_port>
            </Action>
          <Action ID="WaitForStaff" wait_duration="5"/>
        </Sequence>
      </Sequence>
      <Sequence>
        <Action ID="NavigateAroundObstacle"/>
        <Action ID="Alert" />
        <Action ID="BackToSafe" />
      </Sequence>
    </Fallback>
  </BehaviorTree>

  <TreeNodesModel>
    <Action ID="Alert">
      <input_port default="alert_type" name="alert_type"/>
    </Action>
    <Action ID="BackToSafe">
    </Action>
    <Action ID="FoundSurvivor"/>
    <Action ID="HaltExploration"/>
    <Action ID="NavigateAroundObstacle"/>
    <Action ID="SendRTStatusUpdate">
      <input_port default="location" name="command"/>
    </Action>
    <Condition ID="isObstructed"/>
  </TreeNodesModel>
</root>

```

CFR: 0/1=0

o3 (21s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <ReactiveFallback name="Root">
      <Sequence name="HandleSurvivor">
        <Condition ID="foundSurvivor"/>
        <Action ID="Alert" alert_type="status_update"/>
        <Action ID="WaitStaff"/>
        <Condition ID="hasStaffArrived"/>
      </Sequence>

      <Sequence name="HandleObstruction">
        <Condition ID="isObstructed"/>
        <Action ID="MoveTo" location="alternate_path"/>
        <Fallback name="RerouteCheck">

```

```

    <Inverter>
      <Condition ID="isObstructed"/>
    </Inverter>
    <Sequence name="RerouteFailed">
      <Action ID="Alert" alert_type="obstruction_failure"/>
      <Action ID="MoveTo" location="safe_location"/>
    </Sequence>
  </Fallback>
</Sequence>

  <Action ID="MoveTo" location="explore"/>
</ReactiveFallback>
</BehaviorTree>
</root>

```

CFR: $4/2 = 2$

Task 5: Autonomous Crop Harvesting

This behavior tree manages an agricultural robot. It begins by moving to the crop field. As it searches, when it detects an apple, it determines whether it is ripe. If the apple is ripe, the robot picks it up and places it inside a chest. If the apple is rotten, the robot picks it up and discards it. During its operation, if the robot detects signs of pest infestation, it immediately alerts the farmer before autonomously applying the necessary treatment. Once the chest reaches full capacity, the robot proceeds to the farm, where it places the chest on the designated table.

Actions: [MoveTo (parameters: location), DetectApple, CheckRipeness, Pick (parameters: object), Place (parameters: object, destination), AssessPestRisk, NotifyFarmer, TreatInfestation (parameters: object), CheckRotten, Throw, CheckChestFull(parameters: chest)]

Reference

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <MoveTo location="field"/>
    <Fallback>
      <Sequence>
        <DetectApple/>
        <CheckRipeness/>
        <Sequence>
          <Pick object="apple"/>
          <Place object="apple" destination="chest"/>
        </Sequence>
      </Sequence>
      <Sequence>
        <DetectApple/>
        <CheckRotten/>
        <Pick object="rotten_apple"/>
        <Throw/>
      </Sequence>
    </Fallback>
    <Sequence>
      <AssessPestRisk/>
      <NotifyFarmer/>
      <TreatInfestation object="apple"/>
    </Sequence>
    <Sequence>
      <CheckChestFull chest="chest"/>
      <MoveTo location="farm"/>
      <Place chest="ripe_chest" destination="table"/>
    </Sequence>
  </Sequence>
</BehaviorTree>
</root>

```

Ours (5.5s)

```

<root main_tree_to_execute="MainTree" BTCPP_format="4">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <MoveTo location="Crop Field"/>
    <Fallback>
      <Sequence>
        <DetectApple/>
        <CheckRipeness/>
        <Sequence>
          <Pick object="apple"/>
          <Place object="apple" destination="chest"/>
        </Sequence>
      </Sequence>
    </Sequence>
  </Sequence>

```



```

    <Sequence>
      <DetectApple/>
      <CheckRotten/>
      <Pick object="rotten apple"/>
      <Dispose/>
    </Sequence>
  </Fallback>
</Sequence>
<Sequence>
  <AssessPestRisk/>
  <NotifyFarmer/>
  <TreatInfestation object="pest"/>
</Sequence>
<Sequence>
  <CheckChestFull chest="chest"/>
  <MoveTo location="Farm"/>
  <Place chest="chest" destination="table"/>
</Sequence>
</Sequence>
</BehaviorTree>
</root>

```

CFR: 4/1 = 4

BTGenBot (1m 26s)

```

<root main_tree_to_execute = "MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Sequence>
        <Action ID="MoveTo" goal="crop_field"/>
      </Sequence>
      <Sequence>
        <Action ID="Detect" mode="pose" pose="apple"/>
      </Sequence>
      <Sequence>
        <Condition ID="CheckRipeness" target="apple"/>
        <Action ID="Pick" target="apple"/>
        <Action ID="Place" target="chest" pose="position" parameters="0.5;-0.5;0.5;-0.25"/>
      </Sequence>
      <Sequence>
        <Action ID="AssessPestRisk" risk="moderate"/>
      </Sequence>
      <Sequence>
        <Condition ID="CheckChestFull" chest="chest"/>
        <Sequence>
          <Condition ID="CheckChestFull" chest="chest"/>
          <Action ID="Place" target="chest" pose="position" parameters="0.5;-0.5;0.5;-0.25"/>
          <Action ID="TreatInfestation" object="chest"/>
        </Sequence>
      </Sequence>
    </BehaviorTree>
    <Action ID="MoveTo" goal="crop_field"/>
    <Action ID="Detect" mode="pose" pose="apple"/>
    <Action ID="Pick" target="apple"/>
    <Action ID="Place" target="chest" pose="position" parameters="0.5;-0.5;0.5;-0.25"/>
    <Action ID="TreatInfestation" object="chest"/>
  </root>

```

CFR: 4/(0+eps) $\rightarrow \infty$

o3 (18s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="RootSequence">
      <MoveTo location="crop_field"/>
      <RepeatUntilSuccess name="HarvestLoop">
        <Sequence name="HarvestSequence">
          <ForceSuccess>
            <Sequence name="PestHandling">
              <AssessPestRisk/>
              <NotifyFarmer/>
              <TreatInfestation object="pest_area"/>
            </Sequence>
          </ForceSuccess>
          <DetectApple/>
          <Fallback name="AppleProcessing">
            <Sequence name="RipePath">
              <CheckRipeness/>

```

```

    <Pick object="apple"/>
    <Place object="apple" destination="chest"/>
  </Sequence>
  <Sequence name="RottenPath">
    <CheckRotten/>
    <Pick object="apple"/>
    <Throw/>
  </Sequence>
</Fallback>
<Inverter>
  <CheckChestFull chest="harvest_chest"/>
</Inverter>
</Sequence>
</RepeatUntilSuccess>
<MoveTo location="farm"/>
<Place object="chest" destination="table"/>
</Sequence>
</BehaviorTree>
</root>

```

CFR: 4/1=4

Task 6: Automated Assembly and Inspection

This behavior tree manages a robotic arm in a manufacturing facility with the goal of assembling components. Initially, it picks two components from the conveyor roller. Before assembling them, the integrity of each one must be checked. If it's damaged, the damaged component is placed back on the conveyor roller and the arm picks a new component. In this case, it also alerts the staff. Finally, it assembles the two components and places the newly assembled component in the conveyor roller.

Actions: [Pick (parameters: component), Place (parameters: component, destination), CheckIntegrity (parameters: component), isDamaged, AlertStaff, Assemble(parameters: first_component, second_component)]

Reference

```

<root BTCPP_format="4" main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Parallel success_count="2" failure_count="1">
        <Pick component="{first_component}" />
        <Pick component="{second_component}" />
      </Parallel>
      <Fallback>
        <Sequence>
          <CheckIntegrity component="{first_component}" />
          <isDamaged />
          <AlertStaff/>
          <Place component="{first_component}" destination="conveyor_roller" />
          <Pick component="{first_component}" />
        </Sequence>
        <Sequence>
          <CheckIntegrity component="{second_component}" />
          <isDamaged />
          <AlertStaff/>
          <Place component="{second_component}" destination="conveyor_roller" />
          <Pick component="{second_component}" />
        </Sequence>
      </Fallback>
      <Assemble parameters="{first_component}" second_component="{second_component}" />
      <Place component="{second_component}" destination="conveyor_roller" />
    </Sequence>
  </BehaviorTree>
</root>

```

Ours (4.5s)

```

<root BTCPP_format="4" main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Parallel success_count="2" failure_count="1">
        <Pick component="{component1}" />
        <Pick component="{component2}" />
      </Parallel>
      <Fallback>
        <Sequence>
          <CheckIntegrity component="{component1}" />

```

```

        <isDamaged />
        <Place component="{component1}" destination="conveyor_roll" />
    </Sequence>
</Sequence>
    <CheckIntegrity component="{component2}" />
    <isDamaged />
    <Place component="{component2}" destination="conveyor_roll" />
</Sequence>
</Fallback>
<Assemble parameters="first_component" second_component="second_component" />
<Place component="{second_component}" destination="conveyor_roll" />
</Sequence>
</BehaviorTree>
</root>

```

CFR: 2/1=2

BTGenBot (1m 36s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="ComponentManagement">
      <Action ID="Pick" name="PickComponents" parameters="component1, component2"/>
      <Action ID="Place" name="PlaceComponents" parameters="component1, destination, first_component"/>
      <Sequence name="ComponentIntegrityCheck">
        <Action ID="CheckIntegrity" name="CheckComponentIntegrity" parameters="component2, isDamaged"/>
        <Action ID="Place" name="PlaceDamagedComponent" parameters="component2, destination"/>
        <Action ID="Pick" name="PickNewComponent" parameters="component2, component3"/>
      </Sequence>
      <Action ID="Assemble" name="AssembleComponents" parameters="component3, component2"/>
      <Action ID="Place" name="PlaceNewComponent" parameters="component3, destination"/>
    </Sequence>
  </BehaviorTree>
  <TreeNodesModel>
    <Action ID="Assemble">
      <input_port name="component3"/>
      <input_port name="component2"/>
    </Action>
    <Action ID="CheckIntegrity">
      <input_port name="component"/>
      <input_port name="isDamaged"/>
    </Action>
    <Action ID="Place">
      <input_port name="destination"/>
      <input_port name="first_component"/>
    </Action>
    <Action ID="Pick">
      <input_port name="component1"/>
      <input_port name="component2"/>
    </Action>
    <Condition ID="isDamaged">
      <input_port name="component"/>
    </Condition>
    <Action ID="AlertStaff">
      <input_port name="text"/>
    </Action>
  </TreeNodesModel>
</root>

```

CFR: 0/(1+eps)=0

o3 (41s)

```

<root main_tree_to_execute="MainTree">

  <BehaviorTree ID="MainTree">
    <Sequence>
      <SubTree ID="AcquireComponent" component="{component1}"/>
      <SubTree ID="AcquireComponent" component="{component2}"/>
      <Action ID="Assemble" first_component="{component1}" second_component="{component2}"/>
      <Action ID="Place" component="assembled_component" destination="conveyor_roller"/>
    </Sequence>
  </BehaviorTree>

  <BehaviorTree ID="AcquireComponent">
    <RepeatUntilSuccess>
      <Sequence>
        <Action ID="Pick" component="{component}"/>

```

```

    <Action ID="CheckIntegrity" component="{component}"/>
  <Fallback>
    <ForceFailure>
      <Sequence>
        <Condition ID="isDamaged"/>
        <Action ID="Place" component="{component}" destination="conveyor_roller"/>
        <Action ID="AlertStaff"/>
      </Sequence>
    </ForceFailure>
    <Inverter>
      <Condition ID="isDamaged"/>
    </Inverter>
  </Fallback>
</Sequence>
</RepeatUntilSuccess>
</BehaviorTree>

</root>

```

CFR: 2/1=2

Task 7: Household Cleaning

This behavior tree manages a household cleaning robot. First of all, it checks if the room is cleaned or not. If it's cleaned it goes to another room, if it's dirty it starts cleaning. When it identifies a misplaced object it picks it up and places it in the right position. If it sees dust and debris it uses the vacuum cleaner. It does this until the room is cleaned and the next goes to the next room.

Actions: [Pick (parameters: object), Place (parameters: object, location), MoveTo (parameters: location), isCleaned, isDusty, isMisplaced, UseVacuumCleaner]

Reference

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Fallback name="cleaning_or_dusting">
        <Sequence>
          <isCleaned/>
          <MoveTo location="next_room"/>
        </Sequence>
        <Sequence>
          <isDirty/>
          <Pick object="dirty_object"/>
          <Place object="dirty_object" location="cleaning_area"/>
        </Sequence>
      </Fallback>
      <Fallback name="dusting_or_object_placement">
        <Sequence>
          <isDusty/>
          <RepeatUntilSuccess>
            <UseVacuumCleaner/>
          </RepeatUntilSuccess>
        </Sequence>
        <Sequence>
          <isMisplaced/>
          <Pick object="misplaced_object"/>
          <Place object="misplaced_object" location="original_location"/>
        </Sequence>
      </Fallback>
    </Sequence>
  </BehaviorTree>
</root>

```

Ours (4.4s)

```

<root main_tree_to_execute="MainTree" BTCPP_format="4">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <Fallback name="cleaning_or_dusting">
        <Sequence>
          <isCleaned/>
          <MoveTo location="next_room"/>
        </Sequence>
        <Sequence>
          <isDirty/>
          <Pick object="{object}"/>
          <Place object="{object}" location="original_location"/>
        </Sequence>
      </Fallback>
    </Sequence>
  </BehaviorTree>
</root>

```

```

    </Sequence>
  </Fallback>
</Fallback>
  <Inverter>
    <isDusty/>
  </Inverter>
  <Inverter>
    <isMisplaced/>
  </Inverter>
  <RepeatUntilSuccess>
    <UseVacuumCleaner/>
  </RepeatUntilSuccess>
</Fallback>
</Sequence>
</BehaviorTree>
</root>

```

CFR: 4/2=2

BTGenBot (48s)

```

<root main_tree_to_execute = "MainTree" >

  <BehaviorTree ID="MainTree">
    <Fallback name="root_Fallback">
      <Sequence name="clean_room">
        <Fallback name="room_clean">
          <Condition ID="RoomCleanCondition"/>
          <Action ID="GoToNextRoomAction"/>
        </Fallback>
        <Repeat num_cycles="-1">
          <Fallback name="room_dirty">
            <Condition ID="RoomDirtyCondition"/>
            <Sequence name="clean_room_sequence">
              <Action ID="DetectMisplacedObjectAction"/>
              <Action ID="PickUpMisplacedObjectAction"/>
              <Action ID="PlaceObjectAction"/>
            </Sequence>
          </Fallback>
        </Repeat>
      </Sequence>
      <Action ID="GoToNextRoomAction"/>
    </Fallback>
  </BehaviorTree>

  <TreeNodesModel>
    <Action ID="DetectMisplacedObjectAction"/>
    <Condition ID="RoomCleanCondition"/>
    <Condition ID="RoomDirtyCondition"/>
    <Action ID="PickUpMisplacedObjectAction"/>
    <Action ID="PlaceObjectAction"/>
    <Action ID="GoToNextRoomAction"/>
    <Action ID="VacuateAction"/>
  </TreeNodesModel>

</root>

```

CFR: 2/2=1

o3 (26s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Repeat num_cycles="-1">
      <Fallback name="RoomHandler">
        <Sequence name="RoomCleaned">
          <isCleaned/>
          <MoveTo location="next_room"/>
        </Sequence>

        <Sequence name="CleanRoom">
          <Fallback name="CleaningTasks">
            <!-- Handle misplaced objects -->
            <Sequence name="HandleMisplacedObject">
              <isMisplaced/>
              <Pick object="item"/>
              <Place object="item" location="proper_place"/>
            </Sequence>

```

```

    <Sequence name="HandleDust">
      <isDusty/>
      <UseVacuumCleaner/>
    </Sequence>
  </Fallback>
</Sequence>
</Fallback>
</Repeat>
</BehaviorTree>
</root>

```

CFR: 3/2 = 1.5

Task 8: Customer Support and Restocking

This behavior tree manages a retail service robot that handles store inventory and customer assistance. First of all, it explores the store looking for customers in need of help. If it finds one, it asks for the product and it guide the customer to the right shelf. In the meantime, it checks for stock levels on the shelves. If restocking is needed it goes to the warehouse, picks the necessary products and restocks the shelf.

Actions: [Pick (parameters: product), Place (parameters: product, shelf), CheckCustomers, needRestocking, MoveTo (parameters: location), AskCustomer (parameters: product)]

Reference

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Fallback name="main_fallback">
      <Sequence name="search_and_help">
        <CheckCustomers/>
        <MoveTo location="{customer}"/>
        <AskCustomer product="requested_product"/>
      </Sequence>
    <Fallback>
      <Sequence>
        <needRestocking/>
        <MoveTo location="warehouse"/>
        <Pick product="{product}"/>
        <Place product="{product}" shelf="shelf"/>
      </Sequence>
    </Fallback>
  </Fallback>
</BehaviorTree>
</root>

```

Ours (5.9s)

```

<root main_tree_to_execute="MainTree" BTCPP_format="4">
  <BehaviorTree ID="MainTree">
    <Fallback name="main_fallback">
      <Sequence name="search_and_help">
        <Condition ID="IsCustomerNeedHelp"/>
        <Action ID="ExploreStore"/>
        <Action ID="AskCustomer"/>
      </Sequence>
    <Fallback name="restock_fallback">
      <Sequence name="restock_sequence">
        <Condition ID="IsProductAvailable"/>
        <Action ID="Pick (product)"/>
        <Action ID="Place (product, shelf)"/>
      </Sequence>
    <Sequence name="warehouse_sequence">
      <Condition ID="IsWarehouseAvailable"/>
      <Action ID="MoveTo"/>
      <Action ID="Pick (product)"/>
      <Action ID="Place (product, shelf)"/>
    </Sequence>
  </Fallback>
</Fallback>
</BehaviorTree>
</root>

```

CFR: 3/2=1.5

BTGenBot (2m 4s)

```

<root main_tree_to_execute="BehaviorTree">
  <!-- //////////// →
  <BehaviorTree ID="BehaviorTree">
    <Root>
      <Fallback>
        <Sequence>
          <Fallback>
            <Sequence>
              <Action ID="GetCustomer" />
              <Action ID="ApproachCustomer" />
              <Action ID="AskForProduct" />
              <Action ID="GoToShelf" />
            </Sequence>
          </Fallback>
          <Fallback>
            <Sequence>
              <Action ID="GoToShelves" />
              <Fallback>
                <Sequence>
                  <Action ID="CheckShelves" />
                  <ForceSuccess>
                    <Sequence>
                      <Action ID="GoToWarehouse" />
                      <Action ID="FindProduct" />
                      <Action ID="GoToShelves" />
                    </Sequence>
                  </ForceSuccess>
                </Sequence>
              </Fallback>
            </Sequence>
          </Fallback>
        </Sequence>
      </Fallback>
    </Root>
  </BehaviorTree>
  <!-- //////////// →
  <TreeNodesModel>
    <Action ID="ApproachCustomer">
      <input_port name="key_name">CUSTOMER_POSITION</input_port>
    </Action>
    <Action ID="CheckShelves">
      <output_port name="shelf_position">SHELF_POSITION</output_port>
      <input_port name="stock_level">STOCK_LEVEL</input_port>
    </Action>
    <Action ID="GetCustomer">
      <input_port name="key_name">CUSTOMER_POSITION</input_port>
    </Action>
    <Action ID="GoToShelves">
      <input_port name="shelf_position">SHELF_POSITION</input_port>
    </Action>
    <Action ID="GoToWarehouse">
      <input_port name="waypoint">WAREHOUSE_WAYPOINT</input_port>
    </Action>
    <Action ID="FindProduct">
      <output_port name="product_id">PRODUCT_ID</output_port>
      <input_port name="search_position">SEARCH_POSITION</input_port>
    </Action>
    <Action ID="AskForProduct">
      <input_port name="product_id">PRODUCT_ID</input_port>
    </Action>
  </TreeNodesModel>
  <!-- //////////// →
</root>

```

CFR: 1/3= 0.33

o3 (15s)

```

<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <RepeatUntilFailure>
      <Fallback name="Assist_or_Restock">
        <Sequence name="Assist_Customer">
          <CheckCustomers/>
          <AskCustomer product="{product}" />
          <MoveTo location="{shelf}" />
        </Sequence>

```

```

<Sequence name="Restock_Shelf">
  <needRestocking/>
  <MoveTo location="warehouse"/>
  <Pick product="{product}"/>
  <MoveTo location="{shelf}"/>
  <Place product="{product}" shelf="{shelf}"/>
</Sequence>
</Fallback>
</RepeatUntilFailure>
</BehaviorTree>
</root>

```

CFR: 2/1= 2

Results

	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	Task 8	Avg
<u>CFR</u>									
Ours	2	3	0.5	1.5	4	2	2	1.5	2,0625
BTGenBot	0	1	0.66	0	→ ∞	0	1	0.33	0,375
o3	0.5	2	3	2	4	2	1.5	2	2,125
<u>Inference Time</u>									
Ours	4.0s	5.5s	4.2s	3.3s	5.5s	4.5s	4.4s	5.9s	4,6625s
BTGenBot	1m 11s	1m 16s	1m 26s	45s	1m 26s	1m 36s	48s	2m 4s	1m 18,625s
o3	15s	25s	40s	21s	18s	41s	26s	15s	25,125s
<u>Action Coherency</u>									
Ours	✓	✓	✓	✓	✗	✓	✓	✓	7 / 8
BTGenBot	✗	✗	✗	✗	✗	✗	✗	✗	0 / 8
o3	✓	✓	✗	✓	✓	✓	✓	✓	7 / 8
<u>XML Correctness</u>									
Ours	✓	✓	✓	✓	✓	✗	✓	✓	7 / 8
BTGenBot	✗	✓	✓	✗	✗	✗	✓	✗	3 / 8
o3	✓	✓	✗	✓	✓	✗	✓	✓	6 / 8
<u>Semantic Correctness</u>									
Ours	✓	✓	✓	✗	✓	✗	✓	✓	6 / 8
BTGenBot	✗	✗	✓	✗	✗	✗	✗	✓	2 / 8
o3	✓	✓	✗	✗	✓	✗	✓	✓	5 / 8
<u>ZSS</u>									
Ours	3	1	9	2	2	4	11	12	5,5
BTGenBot	12	14	17	10	24	18	17	18	16,25
o3	8	9	20	11	18	23	12	5	13,25